

EXPERIMENT 7

Student Name: ARYAN DAHIYA
Branch: AIT-CSE (AIML)
Semester: 4
Subject Name: Database Management System

UID: 24BAI70697
Section/Group: 24AIT_KRG-G2
Subject Code: 24CSH-298

EXPERIMENT 7

Design and Performance Analysis of Materialized Views (SanDisk, JTG, PayPal)

AIM

To design and implement a materialized view and to compare and analyze execution time and performance differences between simple views, complex views, and materialized views, thereby understanding their impact on query optimization and system performance.

OBJECTIVE

1. To create simple views, complex views, and materialized views.
2. To evaluate their performance by comparing query execution time.
3. To highlight the advantages of materialized views in enterprise-level applications.

SOFTWARE REQUIREMENTS

Database Management System:
Oracle Database Express Edition (Oracle XE)
PostgreSQL Database

Database Administration Tool / Client Tool:
Oracle SQL Developer (for Oracle XE)
pgAdmin (for PostgreSQL)

PROBLEM STATEMENT

EXPERIMENT 7

In large-scale enterprise systems, frequent execution of complex queries can significantly affect performance. Organizations require optimized mechanisms to improve query response time without compromising data consistency.

Materialized views help improve performance by storing precomputed query results, reducing the need for repeated complex calculations.

PRACTICAL / EXPERIMENT STEPS

Step 1: Create base tables (Employee, Department, Payroll).

Step 2: Insert sample data into tables.

Step 3: Create a simple view based on a single table.

Step 4: Create a complex view involving joins and aggregation.

Step 5: Create a materialized view storing precomputed results.

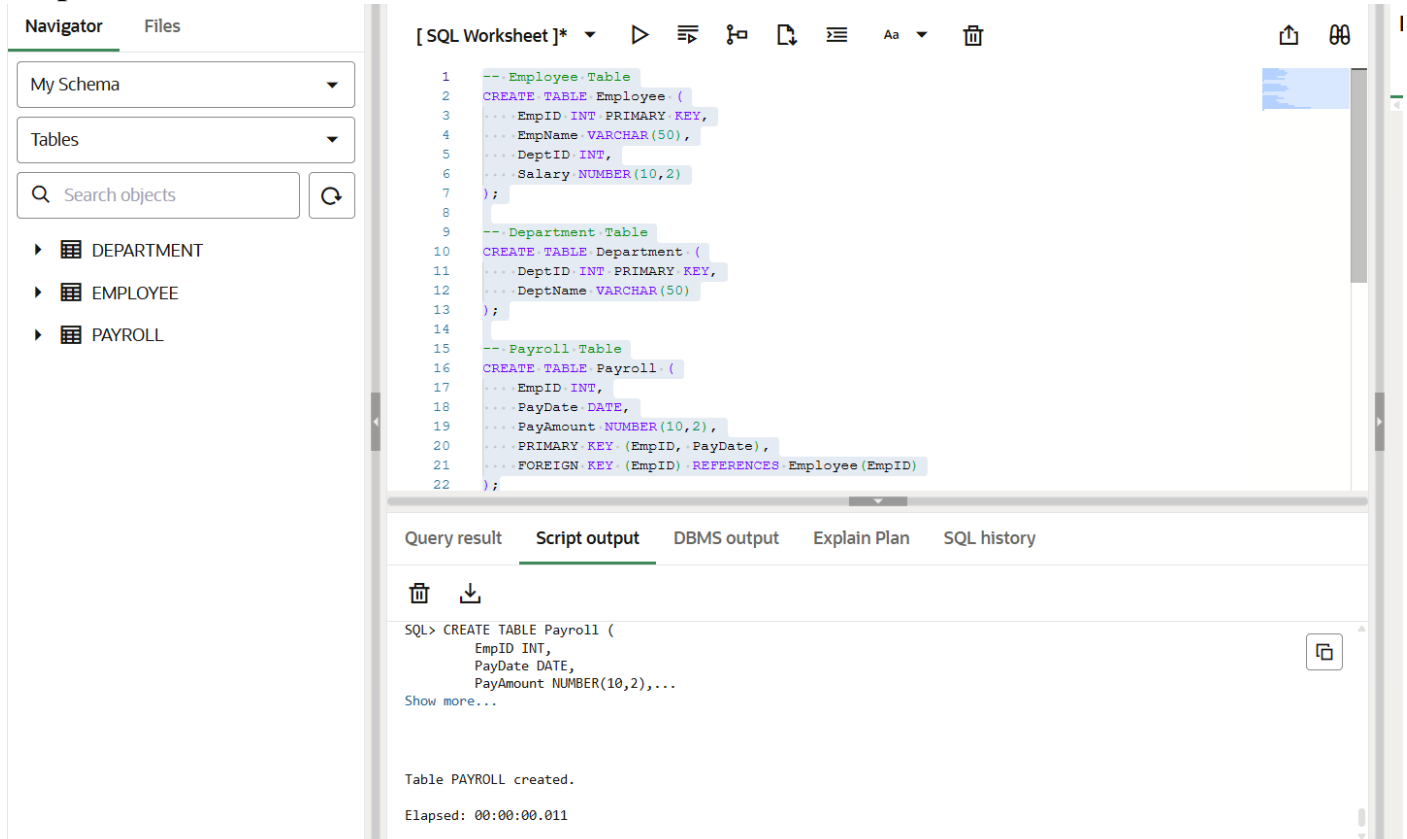
Step 6: Execute queries on all views.

Step 7: Compare execution time and analyze performance.

STEP-WISE OUTPUT

EXPERIMENT 7

Step 1 – Create Base Tables



The screenshot shows the SQL Developer interface with the following components:

- Navigator:** Shows 'My Schema' and 'Tables'. Under 'Tables', there are three tables listed: DEPARTMENT, EMPLOYEE, and PAYROLL.
- SQL Worksheet:** Contains the following SQL code:


```

1  -- Employee Table
2  CREATE TABLE Employee (
3    EmpID INT PRIMARY KEY,
4    EmpName VARCHAR(50),
5    DeptID INT,
6    Salary NUMBER(10,2)
7  );
8
9  -- Department Table
10 CREATE TABLE Department (
11   DeptID INT PRIMARY KEY,
12   DeptName VARCHAR(50)
13 );
14
15 -- Payroll Table
16 CREATE TABLE Payroll (
17   EmpID INT,
18   PayDate DATE,
19   PayAmount NUMBER(10,2),
20   PRIMARY KEY (EmpID, PayDate),
21   FOREIGN KEY (EmpID) REFERENCES Employee (EmpID)
22 );
      
```
- Script output:** Shows the execution of the SQL script. The output indicates that the 'Table PAYROLL created.' and the elapsed time is '00:00:00.011'.

Step 2 – Insert Sample Data

EXPERIMENT 7

Navigator Files

My Schema

Tables

Search objects

- DEPARTMENT
- EMPLOYEE
- PAYROLL

[SQL Worksheet]*

```

23
24
25
26 -- Departments
27 INSERT INTO Department VALUES (1, 'IT');
28 INSERT INTO Department VALUES (2, 'Finance');
29 INSERT INTO Department VALUES (3, 'HR');
30
31 -- Employees
32 INSERT INTO Employee VALUES (101, 'Alice', 1, 70000);
33 INSERT INTO Employee VALUES (102, 'Bob', 2, 80000);
34 INSERT INTO Employee VALUES (103, 'Charlie', 1, 65000);
35 INSERT INTO Employee VALUES (104, 'David', 3, 60000);
36
37 -- Payroll Records
38 INSERT INTO Payroll VALUES (101, TO_DATE('2026-02-01', 'YYYY-MM-DD'), 7000);
39 INSERT INTO Payroll VALUES (102, TO_DATE('2026-02-01', 'YYYY-MM-DD'), 8000);
40 INSERT INTO Payroll VALUES (103, TO_DATE('2026-02-01', 'YYYY-MM-DD'), 6500);
41 INSERT INTO Payroll VALUES (104, TO_DATE('2026-02-01', 'YYYY-MM-DD'), 6000);
42

```

Query result Script output DBMS output Explain Plan SQL history

Elapsed: 00:00:00.002

SQL> INSERT INTO Payroll VALUES (104, TO_DATE('2026-02-01', 'YYYY-MM-DD'), 6000)

1 row inserted.

Elapsed: 00:00:00.001

Step 3 – Create Simple View

EXPERIMENT 7

Navigator Files

My Schema

Tables

Search objects

- DEPARTMENT
- EMPLOYEE
- PAYROLL

[SQL Worksheet]*

```

33 INSERT INTO Employee VALUES (102, 'Bob', 2, 80000);
34 INSERT INTO Employee VALUES (103, 'Charlie', 1, 65000);
35 INSERT INTO Employee VALUES (104, 'David', 3, 60000);
36
37 -- Payroll Records
38 INSERT INTO Payroll VALUES (101, TO_DATE('2026-02-01','YYYY-MM-DD'), 7000);
39 INSERT INTO Payroll VALUES (102, TO_DATE('2026-02-01','YYYY-MM-DD'), 8000);
40 INSERT INTO Payroll VALUES (103, TO_DATE('2026-02-01','YYYY-MM-DD'), 6500);
41 INSERT INTO Payroll VALUES (104, TO_DATE('2026-02-01','YYYY-MM-DD'), 6000);
42
43
44 CREATE VIEW SimpleEmployeeView AS
45 SELECT EmpID, EmpName, Salary
46 FROM Employee;

```

Query result Script output DBMS output Explain Plan SQL history

SQL> CREATE VIEW SimpleEmployeeView AS
SELECT EmpID, EmpName, Salary
FROM Employee

View SIMPLEEMPLOYEEVIEW created.

Elapsed: 00:00:00.016

EXPERIMENT 7

Step 4 – Create Complex View

Navigator **Files**

My Schema

Tables

Q Search objects

- DEPARTMENT
- EMPLOYEE
- PAYROLL

[SQL Worksheet]*

```

49
50 CREATE VIEW ComplexEmployeePayroll AS
51 SELECT e.EmpID, e.EmpName, d.DeptName, SUM(p.PayAmount) AS TotalPaid
52 FROM Employee e
53 JOIN Department d ON e.DeptID = d.DeptID
54 JOIN Payroll p ON e.EmpID = p.EmpID
55 GROUP BY e.EmpID, e.EmpName, d.DeptName;
56

```

Query result **Script output** DBMS output Explain Plan SQL history

SQL> CREATE VIEW ComplexEmployeePayroll AS
SELECT e.EmpID, e.EmpName, d.DeptName, SUM(p.PayAmount) AS TotalPaid
FROM Employee e
JOIN Department d ON e.DeptID = d.DeptID...

Show more...

View COMPLEXEMPLOYEEPAYROLL created.

Elapsed: 00:00:00.015

EXPERIMENT 7

Step 5 – Create Materialized View

Navigator

Files

My Schema

Tables

▶ DEPARTMENT

▶ EMPLOYEE

▶ MATERIALIZEDEMPLOYEEPAYROLL

▶ PAYROLL

[SQL Worksheet]*

▶ ⌵ 🔑 📄 ⌵ Aa 🗑️

55 GROUP BY e.EMPID, e.EMPNAME, d.DEPTNAME;

56

57

58 CREATE MATERIALIZED VIEW MaterializedEmployeePayroll

59 BUILD IMMEDIATE

60 REFRESH COMPLETE ON DEMAND

61 AS

62 SELECT e.EMPID, e.EMPNAME, d.DEPTNAME, SUM(p.PAYAMOUNT) AS TOTALPAID

63 FROM Employee e

64 JOIN Department d ON e.DEPTID = d.DEPTID

65 JOIN Payroll p ON e.EMPID = p.EMPID

66 GROUP BY e.EMPID, e.EMPNAME, d.DEPTNAME;

Query result

Script output

DBMS output

Explain Plan

SQL history

🗑️ ⬇️

SQL> CREATE MATERIALIZED VIEW MaterializedEmployeePayroll
BUILD IMMEDIATE
REFRESH COMPLETE ON DEMAND
AS...
Show more...

Materialized view MATERIALIZEDEMPLOYEEPAYROLL created.

Elapsed: 00:00:00.328

EXPERIMENT 7

Step 6 – Execute Queries on Views

Navigator Files

My Schema

Tables

Search objects

- DEPARTMENT
- EMPLOYEE
- MATERIALIZEDEMPLOYEEPAYROL
- PAYROLL

[SQL Worksheet]*

```

66 GROUP BY e.EMPID, e.EMPNAME, d.DEPTNAME;
67
68
69 -- Simple View Query
70 SELECT * FROM SimpleEmployeeView;
71
72 -- Complex View Query
73 SELECT * FROM ComplexEmployeePayroll;
74
75 -- Materialized View Query
76 SELECT * FROM MaterializedEmployeePayroll;

```

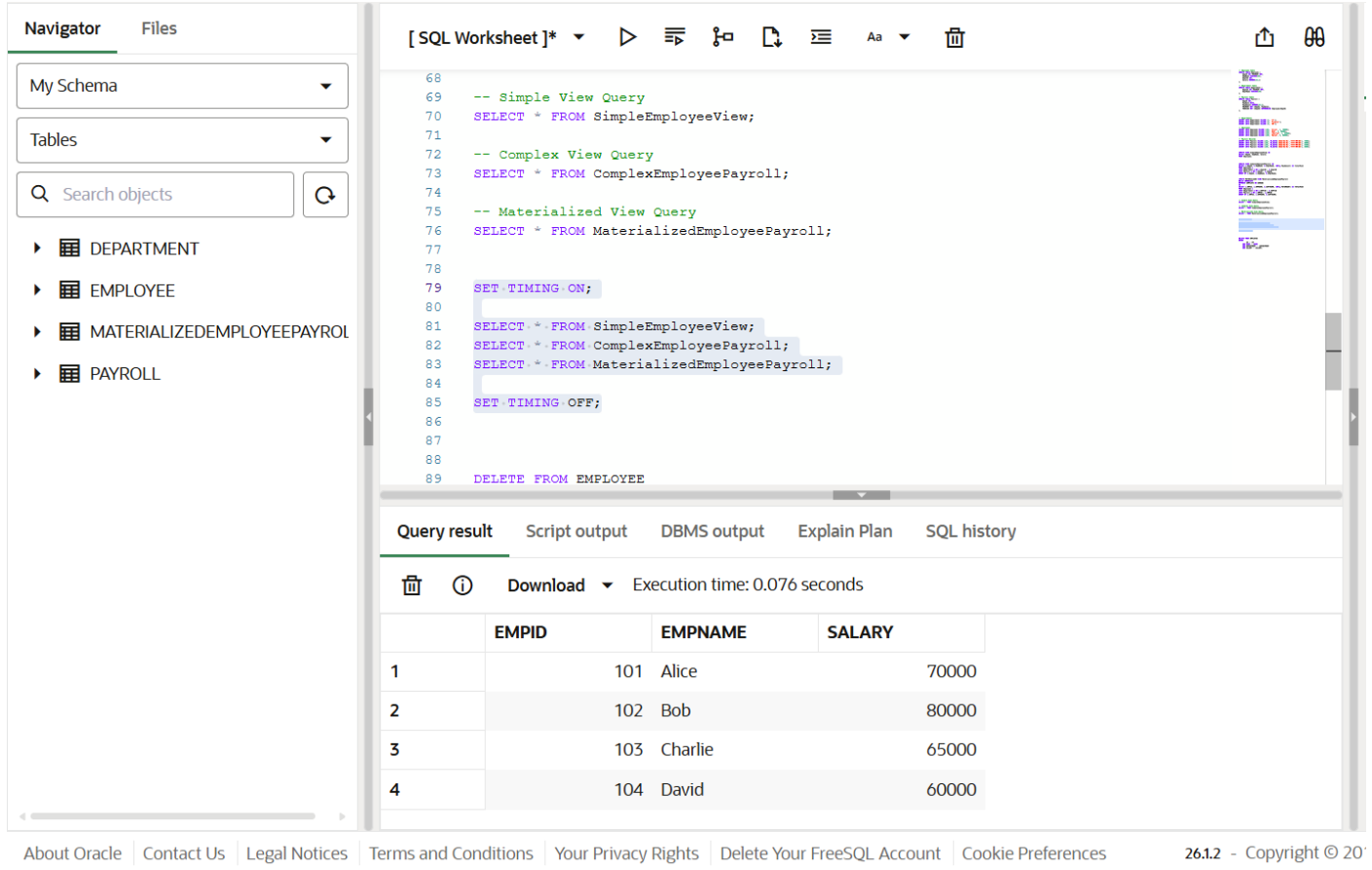
Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.006 seconds

	EMPID	EMPNAME	SALARY
1	101	Alice	70000
2	102	Bob	80000
3	103	Charlie	65000
4	104	David	60000

EXPERIMENT 7

Step 7 – Compare Performance



The screenshot shows the SQL Developer interface. On the left is the Navigator pane showing the schema 'My Schema' with tables: DEPARTMENT, EMPLOYEE, MATERIALIZEDEMPLOYEEPAYROLL, and PAYROLL. The main window displays a SQL worksheet with the following queries:

```

68
69 -- Simple View Query
70 SELECT * FROM SimpleEmployeeView;
71
72 -- Complex View Query
73 SELECT * FROM ComplexEmployeePayroll;
74
75 -- Materialized View Query
76 SELECT * FROM MaterializedEmployeePayroll;
77
78
79 SET TIMING ON;
80
81 SELECT * FROM SimpleEmployeeView;
82 SELECT * FROM ComplexEmployeePayroll;
83 SELECT * FROM MaterializedEmployeePayroll;
84
85 SET TIMING OFF;
86
87
88
89 DELETE FROM EMPLOYEE
  
```

Below the queries, the 'Query result' tab is active, showing the execution time: 0.076 seconds. The result is a table with 4 rows and 4 columns: EMPID, EMPNAME, SALARY, and an unnamed column.

	EMPID	EMPNAME	SALARY	
1	101	Alice	70000	
2	102	Bob	80000	
3	103	Charlie	65000	
4	104	David	60000	

At the bottom of the interface, there are links: About Oracle, Contact Us, Legal Notices, Terms and Conditions, Your Privacy Rights, Delete Your FreeSQL Account, Cookie Preferences, and a footer: 26.1.2 - Copyright © 20

RESULT ANALYSIS

1. Simple View

- Created from a single table.
- Executes quickly for small datasets.
- No performance optimization for complex operations.

2. Complex View

- Includes joins and aggregation.
- Slower than simple view because data is computed every time the query runs.

3. Materialized View

EXPERIMENT 7

- Stores precomputed results physically.
- Significantly faster for repeated queries.
- Requires refresh mechanism to maintain updated data.

REAL-WORLD APPLICATION

Companies such as SanDisk, JTG, and PayPal handle large-scale transactional data.

In such organizations:

- Complex reports run frequently.
- Data warehouses require fast query responses.
- Business intelligence dashboards need optimized performance.

Materialized views are used to:

- Improve reporting speed.
- Reduce server workload.
- Enhance query optimization.
- Improve overall system performance.

LEARNING OUTCOME

After completing this experiment, the learner is able to:

1. Understand the concept and working of materialized views in database systems.
2. Differentiate between simple views, complex views, and materialized views.
3. Create and refresh materialized views in Oracle and PostgreSQL.
4. Measure and compare query execution time.
5. Analyze performance benefits in data-intensive enterprise applications.

CONCLUSION

EXPERIMENT 7

The experiment demonstrates that materialized views significantly improve query performance for repeated complex operations. While simple and complex views calculate results dynamically, materialized views store results physically, reducing computation time.

Therefore, materialized views are highly suitable for enterprise-level systems where performance and efficiency are critical.